

SeuGado — Como rodar o projeto

Instalar, rodar, testar e entregar: o guia de comandos para toda a equipe

27/09/2026

Este guia vale para todo mundo. Siga uma vez para montar o ambiente e depois consulte quando precisar. O seu documento (docs/equipe/<SEU-NOME>.md) diz **o que** implementar; este diz **como rodar**.

1. O que você NÃO precisa rodar

O banco já está pronto e é **um só para a equipe toda**, no Supabase. Não rode nada disto:

Não rode	Por quê
db/migrations/*.sql	Já foram aplicadas pelo Kauê. Rodar de novo quebra o banco de todos
scripts/testar_gee.py	Teste da conta do satélite; já foi feito
Nada no painel do Supabase	Quem administra é o Kauê
Specs em specs/ , kits em revisoes/	São do fluxo do Kauê com o Muse e o Antigravity

Você só **instala as dependências, roda a API e/ou o site, e roda os testes**.

2. Instalar (uma vez)

Ferramenta	Windows (PowerShell)	Mac / Linux
Git	git-scm.com/download/win	já vem, ou brew install git / sudo apt install git
Node 20+ (só quem mexe no site: Ezequiel e Leandro)	nodejs.org → versão LTS	nodejs.org, ou brew install node
uv (Python + pacotes)	powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 iex"	curl -LsSf https://astral.sh/uv/install.sh sh

Depois de instalar o uv, **feche e abra o terminal**. Não precisa instalar Python: o uv baixa o Python 3.12 sozinho na primeira vez.

Confira:

```
git --version
node --version      # v20 ou maior (só front)
uv --version
```

3. Baixar o projeto e criar a sua branch

```
git clone https://github.com/<dono>/seugado.git
cd seugado
git checkout -b feat/<seu-nome>-<assunto>      # ex.: feat/joao-otimizador
```

Sempre rode os comandos **da raiz do repositório** (a pasta que tem o `pyproject.toml`), exceto os do site, que rodam dentro de `frontend/`.

4. Arquivos de ambiente (`.env`)

O Kauê manda os valores em privado. **Nunca commite** `.env` (o Git já ignora).

Raiz do repositório: copie `.env.example` para `.env` e preencha:

Variável	Quem precisa
PYTHONPATH=src (escreva exatamente isso)	todos: é o que faz <code>python -m seugado...</code> e os scripts acharem o código
DATABASE_URL	todos
SEUGADO_TEST_DATABASE_URL	todos (mesmo valor do <code>DATABASE_URL</code>)
SUPABASE_URL, SUPABASE_ANON_KEY	Ezequiel, Leandro
SEUGADO_CORS_ORIGINS	Ezequiel, Leandro: <code>http://localhost:5173</code>
SEUGADO_GEE_PROJECT, SEUGADO_GEE_SERVICE_ACCOUNT_JSON	Leandro (rotina)
TELEGRAM_BOT_TOKEN, TELEGRAM_BOT_USERNAME, TELEGRAM_WEBHOOK_SECRET	Leo (e Leandro, para o link do Telegram)

frontend/.env (Ezequiel e Leandro): copie `frontend/.env.example` e preencha `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY` e `VITE_API_URL=http://localhost:8000`.

Regra de ouro: o Python **não lê o** `.env` **sozinho**. Todo comando `uv run` leva `--env-file .env`. Sem isso, a API dá erro 500 e os testes de banco são pulados sem avisar.

5. Comandos do dia a dia

Backend (Python), na raiz

Para quê	Comando
Instalar/atualizar dependências (rode depois de todo <code>git pull</code>)	<code>uv sync --group dev</code>
Subir a API local	<code>uv run --env-file .env uvicorn seugado.api.main:app --app-dir src --reload</code>

Para quê	Comando
Ver e testar as rotas	abra <code>http://localhost:8000/docs</code>
Rodar todos os testes	<code>uv run --env-file .env pytest</code>
Rodar só os seus testes	<code>uv run --env-file .env pytest tests/planner</code> (troque a pasta)
Lint (obrigatório antes do PR)	<code>uv run ruff check .</code>
Corrigir formatação	<code>uv run ruff format .</code>
Checar tipos (recomendado)	<code>uv run mypy</code>

Frontend (site), dentro de `frontend/`

Para quê	Comando
Instalar dependências (depois de todo <code>git pull</code>)	<code>npm install</code>
Subir o site local	<code>npm run dev</code> → <code>http://localhost:5173</code>
Checar se compila (obrigatório antes do PR)	<code>npm run build</code>

Para usar o site local, a API também precisa estar rodando, em outro terminal.

6. Entrar no site e testar a API com login

1. **Conta:** o Kauê cria a sua conta de admin e manda o e-mail e a senha. Não existe “criar conta”.
2. **Site:** `http://localhost:5173` → entre com a sua conta.
3. **Fazenda de teste:** crie a **sua** em “Clientes e fazendas”, ou pelo `/docs` (`POST /clientes`, depois `POST /fazendas`). Use nomes como `Teste Ezequiel`. O banco é compartilhado: **não mexa nas fazendas dos outros**.
4. **Token para o `/docs`:** as rotas pedem o cabeçalho `authorization: Bearer <token>`. Para pegar o token, faça uma das duas:
 - No site logado, abra *DevTools (F12)* → *Application* → *Local Storage* → `http://localhost:5173`. A chave começa com `sb-` e termina com `-auth-token`; copie o campo `access_token`.
 - Pelo terminal (PowerShell):

```
$r = Invoke-RestMethod -Method Post `
  -Uri "$env:SUPABASE_URL/auth/v1/token?grant_type=password" `
  -Headers @{ apikey = $env:SUPABASE_ANON_KEY } -ContentType "application/json" `
  -Body '{"email":"voce@exemplo.com","password":"sua-senha"}'
$r.access_token
```

(Mac/Linux: o mesmo com `curl -X POST ... -H "apikey: ..." -d '{"...}"`.)

No `/docs`, cole `Bearer <token>` no campo `authorization` da rota. **O token vence em 1 hora;** depois, pegue outro.

7. Testes que usam o banco

- Os testes que precisam do banco leem `SEUGADO_TEST_DATABASE_URL` e são **pulados** se a variável não existir. Por isso o `--env-file .env` é obrigatório.
- O banco de teste é o **mesmo** da equipe. Nos seus testes:
 - crie sempre a **sua própria** fazenda com um UUID novo (`uuid4()`), sem depender de dados que já existem;
 - não apague nada: a tabela `evento` não aceita `DELETE`, e isso é de propósito;
 - teste que só lê a sua fazenda de teste não interfere em ninguém.

8. Como ver a SUA parte funcionando

Pessoa	Precisa rodar	Como ver funcionando
João	só pytest (não precisa de API, site ou banco para o otimizador)	<code>uv run --env-file .env pytest tests/planner</code> . O teste principal compara <code>gerar_plano(estado_exemplo)</code> com <code>tests/fixtures/plano_exemplo.json</code> . Para <code>persistencia/planos.py</code> , os testes usam o banco
Ezequiel	API + site	Entre no site, escolha a sua fazenda de teste no seletor, abra Mapa , desenhe um piquete e salve. Confira pela API em <code>GET /fazendas/{id}/piquetes</code>
Leandro	API + site	Crie cliente e fazenda em Clientes e fazendas , escolha no seletor, cadastre um lote (precisa de piquete, do Ezequiel ou criado pelo <code>/docs</code>). Rotina: <code>uv run --env-file .env python -m seugado.jobs.ciclo --fazenda <id> --recalcular --sem-satelite --sem-envio</code> (depois da integração)
Leo	só o bot, com polling (e o banco)	Crie o bot Dev no @BotFather, ponha o token no <code>.env</code> e rode <code>uv run --env-file .env python scripts/telegram_polling.py</code> . Mande <code>/ajuda</code> para o bot. Para testar <code>/start</code> , pegue o <code>codigo_vinculo_telegram</code> da sua fazenda de teste (página Fazenda do Leandro, ou <code>GET /fazendas/{id}/telegram</code>)

Enquanto as peças dos outros não chegam na `main`, desenvolva com os arquivos de exemplo em `tests/fixtures/` (`estado_projetado_exemplo.json` e `plano_exemplo.json`), como o seu documento explica.

9. Git: do começo ao PR

```
git pull origin main          # comece o dia assim (e sempre que alguém avisar)
git merge main                # se já estiver na sua branch com commits
uv sync --group dev           # dependências podem ter mudado
# ... trabalha ...
git add <arquivos seus>
git commit -m "feat(<area>): <o que fez>"
git push -u origin feat/<seu-nome>-<assunto>
```

Depois, abra o **Pull Request** para a `main` no GitHub. **Não faça merge sozinho**: o merge é na reunião de segunda (exceção: o PR do Leandro com `/me`, clientes e fazendas, que o Kauê faz na hora).

Antes do PR, os três têm de passar: `uv run ruff check .`, `uv run --env-file .env pytest` e, se mexeu no site, `npm run build`.

10. Problemas comuns

Sintoma	Causa	Solução
API responde 500 e o log mostra <code>KeyError: 'DATABASE_URL'</code>	faltou <code>--env-file .env</code>	use o comando da seção 5
<code>ModuleNotFoundError: No module named 'seugado'</code>	faltou <code>--app-dir src</code> ou não está na raiz	rode da raiz, com o comando exato
Testes de banco aparecem como “skipped”	faltou <code>--env-file .env</code>	idem
Site: erro de CORS no console	API sem <code>SEUGADO_CORS_ORIGINS</code> ou fora do ar	confira o <code>.env</code> da raiz e se a API está rodando
API responde 401	token vencido (1 h) ou sem <code>Bearer</code>	pegue outro token
API responde 404 “Fazenda não encontrada”	id errado	copie o id de <code>GET /fazendas</code>
Seletor de fazenda vazio	a rota <code>/fazendas</code> do Leandro ainda não está na <code>main</code>	<code>git pull origin main</code> depois do aviso dele
Windows: <code>npm</code> bloqueado (“execução de scripts desabilitada”)	política do PowerShell	use <code>npm.cmd install</code> / <code>npm.cmd run dev</code> , ou rode no Prompt de Comando
<code>uv</code> não é reconhecido	terminal aberto antes da instalação	feche e abra o terminal
Telegram: o bot não responde no polling	webhook ativo no mesmo bot	use o bot Dev ; o script chama <code>deleteWebhook</code> antes

Travou por mais de 20 minutos? Mande no grupo o comando, a mensagem de erro completa e o que você já tentou.